

Performance Test

Test Run ID: 7a688b6b-48b0-4ca5-84b9-2e958b971942

This chapter provides a summary of the test run metrics. The tables contain the aggregated values of the metrics for the entire test run.

Date	05 June 2026	Max VUs Tested	1,000
Executor	ramping-arrival-rate	Tool	k6

1. Executive Summary

The target endpoint for this performance test is the "https://quickpizza.grafana.com" URL, which is accessed via a GET request with a random query parameter. The testing scenario is a spike test, which involves a ramping arrival rate to simulate a sudden increase in traffic. The load profile consists of a gradual increase from 5 requests per second (RPS) over 3 minutes to 200 RPS for 1 minute, followed by a sustained load of 200 RPS for 5 minutes, and finally a decrease back to 5 RPS for 5 minutes, with a total of 50 pre-allocated Virtual Users (VUs) and a maximum of 1000 VUs.

Stable Threshold	Degradation Onset	Critical Point	Peak Throughput
0-209 TPS	N/A	N/A	200-209 TPS

2. Throughput Analysis

TPS Interval Analysis

TPS Bucket	Total Requests	Failed Requests	Error Rate %
0-9 TPS	1,913	51	2.67%
10-19 TPS	58	0	0.00%
20-29 TPS	196	0	0.00%
30-39 TPS	216	0	0.00%
40-49 TPS	272	0	0.00%
50-59 TPS	335	0	0.00%
60-69 TPS	390	0	0.00%
70-79 TPS	451	0	0.00%
80-89 TPS	507	0	0.00%
90-99 TPS	566	0	0.00%
100-109 TPS	623	0	0.00%
110-119 TPS	685	0	0.00%
120-129 TPS	741	0	0.00%
130-139 TPS	1,080	0	0.00%
140-149 TPS	578	0	0.00%
150-159 TPS	919	0	0.00%
160-169 TPS	1,313	0	0.00%
170-179 TPS	1,053	0	0.00%
180-189 TPS	1,111	0	0.00%
190-199 TPS	23,870	0	0.00%
200-209 TPS	37,284	0	0.00%

3. Performance Summary

Trends

metric	avg	max	med	min	p90	p95	p99
http_req_duration	278.49ms	109401.05ms	231.07ms	0.00ms	241.78ms	244.93ms	-
http_req_waiting	274.35ms	109400.93ms	230.85ms	0.00ms	241.41ms	244.38ms	-
http_req_connecting	1.08ms	49661.39ms	0.00ms	0.00ms	0.00ms	0.00ms	-
iteration_duration	260.54ms	60004.49ms	231.20ms	212.98ms	241.97ms	245.17ms	-

Counters

metric	count	rate
data_received	240.77 MB	251.73 kB/s
data_sent	5.36 MB	5.60 kB/s
http_reqs	74,161	75.72/s
iterations	74,161	75.72/s

Rates

metric	rate
checks_succeeded	99.93% (74110/74161)
checks_failed	0.07% (51/74161)
http_req_failed	0.07% (51/74161)

Gauges

metric	value
vus	68 (min: 0, max: 69)
vus_max	70 (min: 50, max: 70)

Checks

Check Name	Passed	Failed	Success Rate	Status
status is 200	74110	51	99.93%	Fail

4. Advanced Analysis

4.1 Error Analysis

- **Network/Connection errors:**

- Database Fetch Error: server closed the connection unexpectedly
 - + This error indicates that the database server terminated abnormally before or while processing the request.
 - + This error occurred 51 times during the test run.

- **Application errors:**

- None reported in the filtered error logs.

- **Infrastructure errors:**

- None reported in the filtered error logs.

4.2 Observation

The observed errors and time-series metric abnormalities suggest that the database connection was terminated abnormally, leading to a Database Fetch Error. This error occurred 51 times during the test run, indicating a significant issue with the database connection.

Upon further analysis, it appears that the database connection was terminated due to a server crash or an unexpected shutdown. This could be caused by various factors such as:

- Insufficient resources (e.g., memory, CPU) leading to a server crash.
- A bug in the database server code causing it to terminate abnormally.
- A network issue causing the connection to be terminated.

The time-series metric abnormalities in the `http\_req\_duration` and `http\_req\_waiting` metrics suggest that the database connection was terminated during the test run, leading to a significant increase in the average and maximum duration of HTTP requests.

4.3 Final Recommendations

1. **Advanced Connection Management:** Implement a connection pooling mechanism to manage database connections more efficiently. This can help reduce the number of connections and improve performance.
2. **Session Persistence sticky affinity/Redis cache:** Implement session persistence using sticky affinity or Redis cache to reduce the number of database queries and improve performance.
3. **Gateway resilience Istio outlier circuit breaker:** Implement an Istio outlier circuit breaker to detect and prevent cascading failures in the gateway.
4. **Observability tracing and standardized log levels:** Implement observability tracing and standardized log levels to improve monitoring and debugging capabilities.

5. **Database read/write splitting and PgBouncer connection pooling:** Implement database read/write splitting and PgBouncer connection pooling to improve database performance and reduce the load on the database server.

4.4 Conclusion

The system failed to meet its performance objectives due to a significant issue with the database connection. The Database Fetch Error: server closed the connection unexpectedly error occurred 51 times during the test run, indicating a critical issue that needs to be addressed.

To resolve this issue, the recommended architectural and database optimization changes should be implemented. These changes will help improve the performance and reliability of the system, ensuring that it meets its performance objectives.